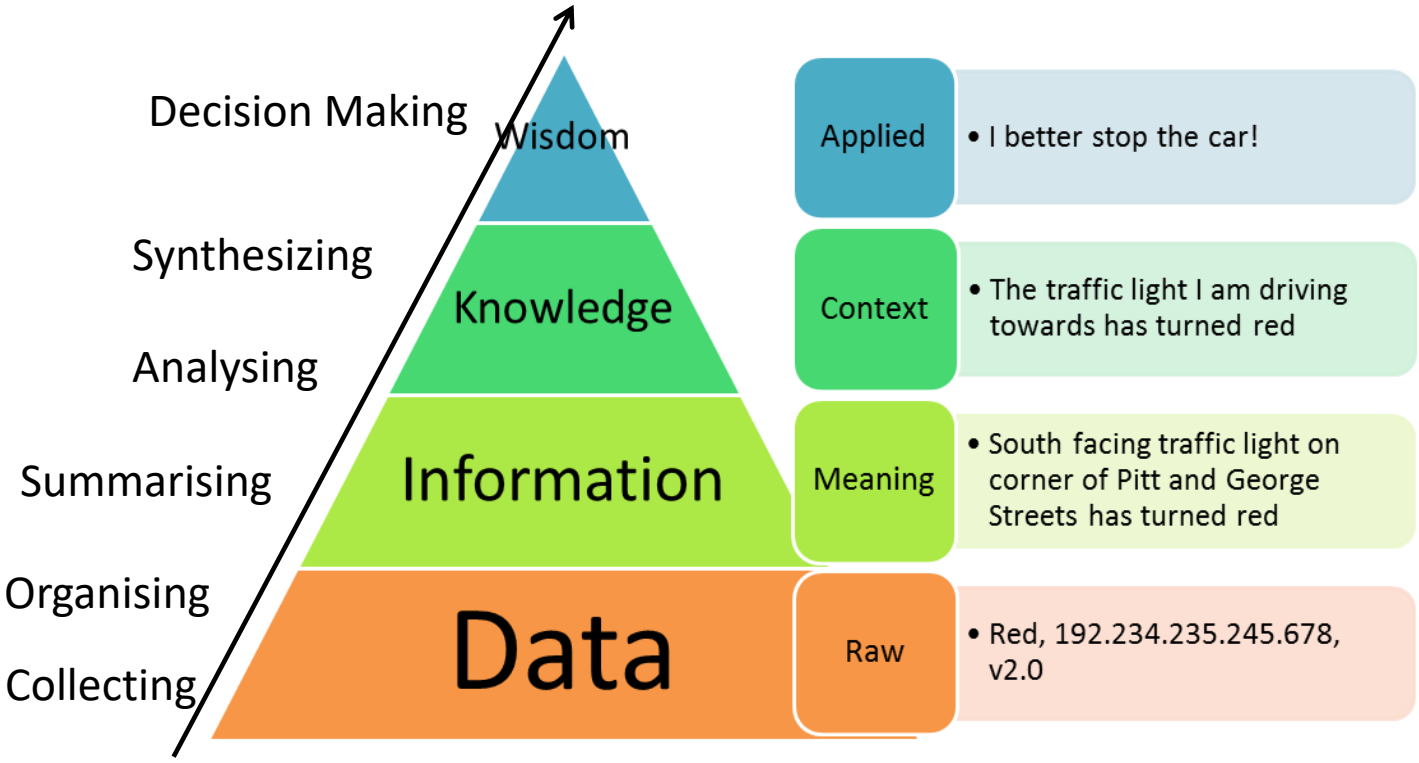
Faktori koji utiču na uspešnost razvoja SBZ:

- **Osnovne ideje**: svaki SBZ bi trebalo da sadrži **mali broj** opštih **osnovnih ideja**
- **Trikovi**: korišćenje **tehnika** koje su karakteristične za **domen**
- **Neizbežna inženjerska rešenja**: dobar **odabir** neizbežnih **inženjerskih rešenja** koja doprinose uniformnosti i upotrebljivosti sistema
- **Manje bitna inženjerska rešenja**

SBZ – Terminologija:

- Podatak (činjenica)
- Informacija (podatak koji ima određeno značenje)
- Znanje (rezultati analize informacija koje se dovode u neki kontekst)
- Mudrost (primenjeno znanje)



© 2011 Angus McDonald

Sistem baziran na znanju je sistem koji se oslanja na bazu znanja, tj. kolekciju znanja preuzetog od ljudi (ne nužno) i skladištenog u sistemu tako da sistem može da rasuđuje (rezonuje) koristeći to znanje. Znanje je vrsta informacije koju ljudi (ne nužno) koriste za rešavanje problema. Platon: znanje je opravdano (obrazloženo) istinito uverenje. Ova definicija je začetak tzv. tripartitnog pogleda na znanje (istinitost, uverenost, opravdanost). Krajem 20. veka znanje se definiše kao neka vrsta informacije.

Znam *p* ako:

- Iskaz *p* je tačan (istinitost)
- Verujem da je *p* tačno (uverenost)
- Moje verovanje da je *p* tačno je opravdano (opravdanost)

Istinitost: Ako smatram da je iskaz *p* tačan (uverenje) i postoji obrazloženje (opravdanost) zašto bi *p* bio tačan, a *p* nije tačan (istinitost), onda svakako ne znam *p*.

Uverenost: Ukoliko je *p* tačno (istinitost) i obrazloženo (opravdanost), ali ja nisam uveren u *p* (uverenje), ne znam *p*.

Opravdanost: Ako je *p* tačno (istinitost) i ja definišem moje obrazloženje zašto je *p* tačno (uverenje), ali obrazloženje je neopravdano (opravdanost), onda ne znam *p*.

Sedam karakteristika koncepta znanja:

1. Znanje ima praktičan aspekt - pomaže pri rešavanju problema
2. Znanje jeste ili nije personalno ograničeno – personalizovano ili (eksterno) reprezentovano
3. Znanje ima normativnu strukturu - trajno važeći paterni (norme) očekivanog ponašanja
4. Znanje je interno umreženo - entitet koji predstavlja znanje ima svoju internu strukturu (mrežu). Učenje je integracija znanja u postojeću strukturu (mrežu).
5. Znanje je eksterno umreženo - znanje mora da bude povezano sa drugim znanjem da bi bilo znanje. Sistem znanja mora da bude konzistentan.
6. Znanje je dinamično - novo znanje može da obezvredi staro ili da mu poveća vrednost. Znanje se može i zaboraviti, ili može da nestane ako se dugo ne koristi.
7. Znanje ima institucionalne kontekste - produkcija i distribucija znanja su važni za institucije

Znanje može da bude:

- Specifično ili opšte
- Egzaktno ili približno
- Proceduralno ili deklarativno

Komponente znanja:

- **Činjenica**: **stvar koja se zna** ili je dokazano da je istinita.
- **Koncept**: apstrakcija ili **uopštenje iskustva** ili rezultat transformacije postojećih koncepata.
- **Procedura**: **niz akcija koje se sprovode po određenom redosledu i načinu**.
- **Model**: **reprezentacija sistema** ili složenog entiteta.
- **Heuristika**: **iskustveno bazirana tehnika za rešavanje problema**, učenje i otkrivanje kojim se dolazi do **rešenja koje nije garantovano optimalno** ali je dovoljno dobro za zadati skup ciljeva.
- **Primer**: jedna od više stvari, ili **deo nečega** što se **uzima da pokaže karakter celine**.

Sistem baziran na znanju (SBZ) oslanja se na reprezentovano znanje da bi izvršio zadatke. Koristi metode rasuđivanja/zaključivanja da bi izveo novo znanje. Obično je ograničen na specifični problemski domen.

Elementi SBZ:

1) Baza znanja - repozitorijum domenskog znanja i meta znanja
2) Modul za zaključivanje - softverski program koji zaključuje na osnovu znanja u bazi znanja
3) Radna memorija - oslikava trenutno stanje u domenu i to sa činjenicama
4) Agenda - zadovoljena pravila se smeštaju u agendu
5) Modul za obrazlaganje rezonovanja - obezbeđuje objašnjenja postupka rezonovanja
6) Modul za prikupljanje znanja - obogaćuje sistem mogućnostima samoučenja
7) Korisnički interfejs - omogućuje korisnicima da rade u jeziku koji razumeju

Ekspertski sistem je kompjuterski sistem koji:

- Obavlja funkcije/zadatke slične onima koje obično vrši stručnjak (ekspert)
- Koristi reprezentaciju ljudske ekspertize u specijalizovanom domenu radi obavljanja zadataka
- Oslanja se na mehanizam zaključivanja i bazu znanja u kojoj se nalazi znanje stručnjaka

Ekspertski sistem baziran na pravilima je ES čija baza znanja sadrži znanje stručnjaka reprezentovano u formi IF-THEN pravila (IF pretpostavka THEN zaključak/akcija). Tvrdnje u IF-THEN formi se još nazivaju i produkciona pravila. Pravilima se mogu predstavljati: relacije, preporuke, direktive, strategije i heuristike. ES baziran na pravilima preslikava znanje stručnjaka u skup produkcionih pravila koja primenjuje na činjenično stanje podataka iz radne memorije da bi donosio zaključke. Pretpostavka je patern na osnovu kojeg se utvrđuje da li pravilo može da se primeni na posmatrani problem. Sastoji se od dva dela - (lingvistički) objekat i vrednost, povezanih operatorom. Zaključak/Akcija definiše predviđeni korak za rešavanje problema. Izvršene akcije mogu da promene stanje u radnoj memoriji što će dovesti do aktiviranja novih pravila.

Rad modula za zaključivanje - pre početka rezonovanja radna memorija se inicijalizuje sa činjenicama, a baza znanja se popuni pravilima. Pokreće se modul za zaključivanje koji uparuje paterne koji su definisani u pretpostavkama sa činjenicama u radnoj memoriji da bi pronašao pravila koja su zadovoljena. U zadovoljenom pravilu leva strana mora da se slaže sa činjenicama u radnoj memoriji. Modul za zaključivanje određuje 1 ili više pravila čiji su preduslovi zadovoljeni. Zadovoljena pravila se smeštaju u agendu. Samo jedno pravilo može da se aktivira u jednom trenutku. Razrešavanje konflikta određuje koje će se pravilo aktivirati/izvršiti. Izvršavanje pravila:

- Izvršavaju se akcije desne strane odabranog pravila
- Izmene radne memorije (opciono) – generisanje novih ili uklanjanje postojećih činjenica
- Pravilo se uklanja iz agende

Ako je pritom došlo do izmene radne memorije, vrši se uparivanje i ažuriranje agende. Rad modula za zaključivanje prestaje kada u agendi nema više pravila ili eksplicitnom STOP komandom.

Ulančavanje unapred (forward chaining), vođeno podacima (data-driven) je rezonovanje od činjenica ka zaključku. Znamo samo koje je trenutno stanje radne memorije, a nema se pojma kojim putem će krenuti zaključivanje i koji će konačan zaključak biti. U svakom koraku se aktivira (tačno jedno) pravilo najvišeg prioriteta. Pravilo se može aktivirati ako su svi delovi leve strane zadovoljeni. Sve dok postoje, činjenice se koriste za zadovoljavanje levih strana pravila. Proces se zaustavlja kada više nema pravila u agendi koja bi se mogla aktivirati. Često se koristi u ES realnog vremena za nadgledanje i upravljanje. Sakupe se sve informacije i onda se iz njih zaključi sve što može da se zaključi. Često se dešava da aktiviranje pravila nema nikakvu posledicu na planirani rezultat. Neefikasno je ako znamo hipotezu, a želimo da proverimo da li je zadovoljena.

Ulančavanje unazad (backward chaining), vođeno upitom (query-driven) je rezonovanje od zaključka ka činjenicama, tj. potvrđivanje zaključka sa činjenicama. Znamo šta želimo da dokažemo, ali ne znamo da li ćemo to dokazati sa činjenicama iz radne memorije. Željeni cilj se stavlja u radnu memoriju a rezoner (modul za zaključivanje) pokušava da pronađe dokaze da dokaže željeni cilj, polazeći od hipoteze (upita). Prvo se pretražuje baza znanja za pravilima koja bi mogla da dokažu hipotezu. To su pravila čija desna strana je hipoteza. Ako ne postoje takva pravila, hipoteza je oborena. Ako postoje pravila, proverava se da li su leve strane zadovoljene činjenicama. Ako jesu, hipoteza je dokazana. Ako nisu, leva strana postaje podhipoteza i rekurzivno se ponavlja ciklus. Ciklus se ponavlja dok se ne obori hipoteza (neka od podhipoteza ne može da se izvede iz baze znanja) ili se ne dokaže hipoteza (sve njene podhipoteze se izvode iz baze znanja). Često se koristi u dijagnostici i sistemima za konsultovanje.

U situaciji gde se činjenicama zadovoljavaju preduslovi više od jednog pravila, dobija se set pravila koja su zadovoljena. Taj set se zove konflikt set. Metod odabira pravila u ovakvim slučajevima se zove razrešavanje konflikta (conflict resolution). Strategije razrešavanja konflikta:

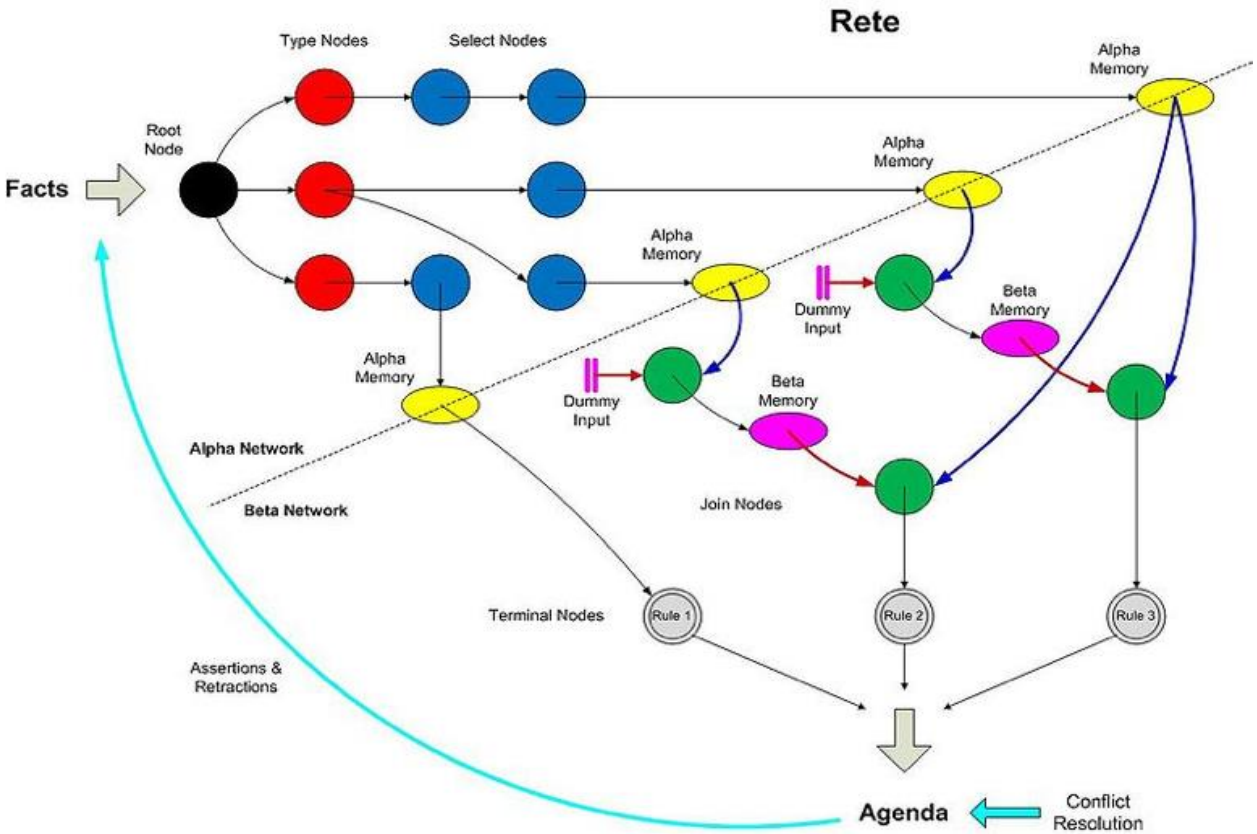
1. Odabira se pravilo najvišeg prioriteta
    - Najčešće kada se pravi baza znanja redosled pravila određuje njihov prioritet
    - Može se eksplicitno zadati prioritet pravilu
    - Pogodno za manje baze znanja (do 100 pravila)
2. Odabira se pravilo na osnovu njegove pozicije
    - Odaberi prvo pravilo koje je definisano pre ostalih u tekstu
3. Odabira se najspecifičnije pravilo
    - Pravilo je specifičnije ako njegovo zadovoljenje zahteva ispunjavanje više preduslova
4. Koriste ste metapravila
    - Pravila o pravilima
    - Na primer, *pravila koja zadaje ekspert imaju više prioritete nego pravila koje zadaje početnik*
5. Najsvežiji podaci
    - Odaberi pravilo koje je zadovoljeno na osnovu činjenica koje su najskorije dodate

Markovljev algoritam - pravila sa većim prioritetima se prvo aktiviraju. Omogućuje efikasnije izvršavanje produkcionih sistema, ali je još uvek nedovoljno efikasan za SBZ sa velikim skupovima pravila.

Rete algoritam provera zadovoljenosti (pattern matching) činjenica za pravila u rule engine-ima. Pravila su skup pretpostavki i akcija. Činjenica je vektor svojstava (tuple, n-torka podataka). Pretpostavke testiraju svojstva činjenica. Rete mrežu definišu usmereni aciklični grafovi koji predstavljaju setove pravila. ES koji koristi Rete algoritam kreira mrežu čvorova (Rete mreža), gde svaki čvor odgovara uslovu/delu uslova iz neke od pretpostavki u levoj strani pravila. Putanja od korenskog čvora ka listu odgovara levoj strani pravila. Svaki čvor ima svoju memoriju u kojoj se nalaze sve činjenice koje zadovoljavaju uslov. Rete mreža smanjuje redundanciju kroz deljenje čvorova u mreži (uslovi se dele na poduslove). Čuva delimična zadovoljavanja uslova, čime se izbegava ponovna evaluacija kompletnih uslova za činjenice. Na efikasan način rukuje činjenicama u memoriji čvorova. Pravila koja zadaju developeri ili eksperti dinamički se prevode u Rete mrežu. Rete mreža se predstavlja korišćenjem mreže memorijskih objekata gde se sa objektima proverava da li svojstva činjenica zadovoljavaju uslove pravila. Za svaku činjenicu kreira se element radne memorije (WME) koji predstavlja tu činjenicu. Svaka činjenica (WME) ulazi u Rete mrežu kroz korenski čvor mreže. Korenski čvor prosleđuje činjenicu svojim child čvorovima. Činjenica zatim prolazi kroz neku od grana stabla dok ne stigne do nekog od krajnjih čvorova (listova).

Alfa mreža: „Leva“ (alfa) strana grafa čvorova formira diskriminacionu mrežu koja je odgovorna za odabir pojedinačnih elemenata radne memorije (WME), odnosno činjenica. Odabir činjenice je na bazi testova uslova koji uparuju atribute WME sa konstantnim vrednostima. Ako se WME uspešno upari sa uslovima predstavljenim jednim čvorom, prosleđuje se sledećem čvoru koji pak testira neki drugi atribut. Svaka grana alfa mreže završava se čvorom koji se zove alfa memorija. Alfa memorija čuva WME-ove (činjenice) koje zadovoljavaju sve uslove predstavljene alfa čvorovima pređene grane alfa mreže.

Beta mreža: „Desna“ (beta) strana grafa prvenstveno izvršava objedinjavanje WME-ova. Beta mreža se sastoji od čvorova sa 2 ulaza, gde svaki čvor ima „levi“ i „desni“ ulaz. Svaki beta čvor šalje izlaz na beta memoriju. Beta čvorovi obrađuju tokene (liste WME-ova, činjenica). Jedan token iz alfa mreže u beta mrežu unosi jedan WME (činjenicu). Beta čvor može da kreira token koji sadrži listu WME-ova koji predstavlja parcijalno poklapanje/match više činjenica. Taj token se smešta u beta memoriju (svaki beta čvor ima svoju memoriju) i prosleđuje dalje child čvorovima u beta mreži. Token koji stigne do kraja mreže je potpuni match činjenice u tokenu sa uslovima jednog pravila. Za svaki token koji stigne do krajnjeg čvora će se aktivirati pravilo predstavljeno tom granom mreže.



Prednosti ES baziranih na pravilima:

- Prirodna reprezentacija znanja - eksperti često svoje znanje iskazuju upravo u obliku pravila
- Uniformna struktura - sva pravila imaju oblik IF-THEN
- Razdvajanje znanja od rezonovanja - pojednostavljuje održavanje baze znanja. Omogućuje da različite aplikacije mogu da koriste isti shell. Omogućuje da se ista baza znanja može koristiti sa različitim tehnikama zaključivanja
- Nepotpuna i neprecizna znanja - ES bazirani na pravilima najčešće omogućuju predstavljanje i korišćenje nepotpunih i nepreciznih znanja

Nedostaci ES baziranih na pravilima:

- Nejasna veza između pravila - teško je ustanoviti kako pojedinačno pravilo učestvuje u celokupnoj strategiji
- Neefikasne strategije pretrage pravila - u slučajevima u kojima je baza znanja velika (preko 100 pravila) ES ume da bude spor jer se pretražuje čitava baza znanja u svakom ciklusu. Posledica: veliki sistemi bazirani na pravilima mogu da budu neadekvatni za real-time aplikacije
- Nemogućnost učenja - proširivanje baze znanja je posao inženjerstva znanja, retko je podržana mogućnost samoučenja

Razlozi za pravljenje ES-a:

- Da **sačuva** znanje nekog eksperta, da bi se obezbedilo osiguranje od situacije kada ekspert ode na drugo mesto, penzioniše se ili umre.
- Da **diseminira** znanje eksperta tako da ono postane dostupno i na mestima gde nije taj ekspert.
- Da **obezbedi ujednačenost** preporuka/odluka.
- Kao **osnova za obuku** drugih specijalista.

Prednosti ES-a nad ekspertima ljudima:

- Znanje je **trajno**
- Znanje se **lako replicira**
- Znanje je **predstavljeno eksplicitno**, i može se evaluirati
- Sistem je **stabilan** – ljudi mogu imaju i loše dane, računari ne.
- Kada se jednom naprave, **troškovi eksploatacije su niski**.

Nedostaci ES-a:

- **Razvoj** ES-a obično **zahteva puno vremena i novaca**
- **Stepen neuspeha** ES projekata je, istorijski gledano, **visok**
- **Čovek-ekspert** može da **unapređuje svoje znanje, ES** to, najčešće, **ne može**

Mikro-svetovi su izolovani domeni znanja koji se mogu posmatrati bez potrebe da se razvija opšta teorija zdravorazumskog znanja. Predmet su izučavanja inženjerstva znanja. U ovakvim oblastima je moguće napraviti SBZ pošto se ne oslanja na opšta pravila i veliku količinu činjeničnih znanja.

Tipovi ekspertize se dele na sledeće mentalne komponente i sposobnosti:

- Komponente
  - Visoka kreativnost
  - Analitičnost
  - Striktna proceduralnost
- Sposobnosti
  - Duboke kognitivne veštine
  - Logičke veštine rasuđivanja
  - Socijalne veštine

Drajfus – fenomenologija usvajanja veština: početnik, napredni početnik, kompetentnost, profesionalnost, ekspertiza, master.

Metode za reprezentaciju znanja (ZR metode):

- Produkciona pravila
- Stabla, grafovi (npr. semantičke mreže, konceptne mape, konceptni grafovi, ontologija)
- Šeme i frejmovi
- Logika

Produkciona pravila se najčešće koriste za predstavljanje znanja u ES. Formalna varijacija je Backus-Naur forma (BNF) - metajezik za definisanje sintakse jezika. Gramatika je kompletan, nedvosmislen skup produkcionih pravila za određeni jezik. Stablo parsiranja je grafička reprezentacija rečenice u tom jeziku. Pravila daju samo sintaksni opis jezika. Osnovni princip produkcionih sistema je da bi **svako pravilo trebalo da bude nezavisan deo znanja** i da **ne bi trebalo da bude svesno drugih pravila**. U tom slučaju modul za zaključivanje može jednostavno da aktivira pravilo svaki put kada su premise pravila zadovoljene. Može se koristiti ulančavanje unapred i ulančavanje unazad kroz pravila. Postoje dva osnovna pitanja pri implementaciji sistema baziranih na pravilima: kako se implementira razrešavanje konflikta i kako se činjenice uparuju sa pravilima?

Prednosti produkcionih pravila:

- Jednostavna i laka za razumevanje
- Moguća direktna računarska implementacija
- Formalne osnove za neke varijante složenijih sistema baziranih na znanju

Nedostaci produkcionih pravila:

- Jednostavne implementacije su veoma neefikasne
- Neki tipovi znanja ne mogu se lako izraziti pravilima
- Veliki skupovi pravila su teški za razumevanje i održavanje

Strukturirani objekti (grafovi i stabla) su pogodna reprezentacija strukturiranog znanja. Koriste se sledeće strukture: semantička mreža, konceptna mapa (nema nasleđivanja) i ontologije (deskriptivna logika).

Semantičke mreže su grafička reprezentacija iskaznih (propozicionih) informacija. Semantička mreža je jednostavna reprezentaciona šema koja za predstavljanje znanja koristi graf označenih čvorova i označenih usmerenih lukova. Obično se koristi za predstavljanje statičkih, taksonomskih rečnika koncepata ili drugih znanja koja se sastoje od koncepata i relacija među konceptima. Grafički prikaz pridružen semantičkoj mreži je važan razlog njihove popularnosti. Semantičke mreže se obično koriste u sprezi sa skupom pristupnih procedura koje obavljaju „rezonovanje“. Semantička mreža se predstavlja kao označeni, usmereni graf. Čvorovi su entiteti, a linkovi relacije. Čvorovi predstavljaju objekte, koncepte ili situacije. Linkovi predstavljaju veze koje sadrže strukturne informacije o znanju koje se predstavlja. Tripleti objekt-atribut-vrednost (OAV) služe za karakterizaciju znanja u semantičkoj mreži. Atributima se izražavaju dodatne informacije o čvorovima u semantičkoj mreži. Relacija AKO (a-kind-of) se koristi da predstavi vezu između klasa i predstavlja specijalnu relaciju nasleđivanja. Relacija IS-A se koristi da predstavi vezu između instance klase sa klasom. HAS-A relacija označava atribut objekta ili klase.

Konceptni grafovi su semantičke mreže za predstavljanje značenja (jednostavnih) rečenica prirodnog jezika. Dva tipa čvorova: konceptni čvorovi (postoje dva tipa koncepata, individualni koncepti i generički koncepti) i relacioni čvorovi (binarne relacije među konceptima).

Problem predstavljanja nebinarnih relacija se rešava tako što se relacija pretvori u objekat (reifikacija - smatranje apstraktnog koncepta za realan).

Zaključivanje nasleđivanjem - jedan od glavnih načina rezonovanja u semantičkim mrežama je nasleđivanje vrednosti duž linkova podklasa i instanci. Semantičke mreže se razlikuju po načinu nasleđivanja višestrukih različitih vrednosti. Nasleđuju se sve moguće vrednosti, ili se nasleđuje samo „najniža“ vrednost ili vrednosti.

Višestruko nasleđivanje - čvor može da ima proizvoljan (konačan) broj superklasa koje ga sadrže kao podklasu, što omogućuje čvoru da nasledi svojstva od više „roditeljskih“ čvorova i njihovih predaka u mreži. Za određivanje nasleđivanja u takvim „zapetljanim“ mrežama gde je dozvoljeno višestruko nasleđivanje se često koriste pravila:

- Ako je X<A<B i oba A i B imaju svojstvo P, tada X nasleđuje to svojstvo od A.
- Ako je X<A i X<B ali nije A<B niti B<A, i A i B imaju svojstvo P sa različitim i nekonzistentnim vrednostima, tada X uopšte ne nasleđuje to svojstvo

Ima više rešenja za probleme izuzetaka i višestrukog nasleđivanja:

- Pravila - uvedemo pravila koja se odnose na konkretne instance
- Udaljenost u stablu nasleđivanja (blizina)
- Višestruka stabla - pretpostavljeno stablo i po jedno za svaku alternativnu hipotezu
- Višestruki čvorovi za isti koncept - mapiranje jednog objekta ili klase na više čvorova
- Evidencijalno rezonovanje - koristi teoriju verovatnoće za manipulisanje neodređenošću i logiku za predstavljanje strukture. Odabira se najverovatniji odgovor. Koraci:
  - Iznošenje činjenica
  - Izračunavanje vrednosti likelihood funkcije za svaku od konkurentnih hipoteza na osnovu činjenica
  - Izbor najizglednije hipoteze

Mreža sa striktnim nasleđivanjem sadrži čvorove (koncepti, svojstva) i usmerene linkove (generalizacija/IS_A relacija).

Implementacija semantičkih mreža:

Direktan pristup

- Kao grafovska struktura
- Zaključuje se prelaskom grafa

Indirektan pristup

- Logičko/funkcionalno programiranje
- Objektno orijentisano programiranje
- Hibridni pristup

Prednosti semantičkih mreža:

- Pogodne su za predstavljanje strukturiranog činjeničnog znanja
- Fleksibilne za predstavljanje konceptnih zavisnosti.
- Mogu da predstave ideje kauzalnosti.
- Relativno lako obuhvataju i predstavljaju rečenice prirodnog jezika.

Nedostaci semantičkih mreža:

- Izražajnost - čvor nema internu strukturu, problem predstavljanja nebinarnih relacija
- Efikasnost - može da dovede do velikih skupova čvorova i linkova
- Upotrebljivost - nedostatak standarda, imenovanje čvorova

Plitko znanje je znanje bazirano na činjenicama bez razumevanja uslova koji dovode do tih činjenica. Semantičke mreže su primer formalizma za predstavljanje plitkog znanja (sve znanje je u čvorovima i vezama). Duboko znanje je znanje zasnovano na formalnoj teoriji domena. Objašnjava zašto se nešto dešava. Šeme su primer formalizma za predstavljanje dubokog znanja.

Šeme su mreže bazičnih gradivnih blokova za organizovanje, čuvanje i pronalaženje znanja. Pogodne su za reprezentovanje složenijih znanja - kauzalne (uzročne) veze između opažaja ili akcije i odgovarajućeg ishoda. Imaju dublje znanje nego semantičke mreže jer čvorovi mogu da imaju internu strukturu. Šema može da se linkuje sa drugim šemama. Šeme mogu da se hijerarhijski organizuju. Najčešće korišćene instance šema su frejmovi i skriptovi (vremenski uređene sekvence frejmova).

Konceptna šema je apstrakcija koja obuhvata opšte/tipične karakteristike konkretnih objekta. Sadrži najvažnija svojstva koja se obično pridružuju objektu tog tipa. Pojednostavljuje rasuđivanje koncentrisanjem na suštinske aspekte.

Frejmovi su grupe imenovanih slotova pomoću kojih se definišu stereotipni objekti. Frejm predstavlja znanje o objektu. Frejmovi se mogu koristiti za predstavu: deklarativnog znanja (objekti i svojstva), proceduralnog znanja (preuzimanje informacija i dostizanje cilja) i nasleđivanja. Frejmovi su organizovani hijerarhijski. Znanje se najčešće organizuje po vezama uzroka i efekta. Slotovi mogu da sadrže razne podatke, ali i druge frejmove. Mogu da sadrže i proceduralne dodatke (procedure koje se pozivaju u specifičnim situacijama u koje je uključen određeni slot). Postoje dve vrste frejmova - frejmovi koji reprezentuju klase (Class Frames) i frejmovi koji reprezentuju individuale ili instance (Instance Frames). Obe vrste su identične u pogledu strukture i konstrukcije sa tim što se frejmovi klasa koriste za opis čitavih klasa, a frejmovi instanci se koriste da opišu individualne objekte ili događaje. Postoje dve vrste rezonovanja nad frejmovima: bottom-up rezonovanje (vođeno podacima ili događajima) i top-down rezonovanje (vođeno očekivanjima).

Struktura frejma sadrži dva osnovna elementa – slotove i facete. Svaki slot sadrži jedan ili više faceta. Najčešće postoje slotovi kojima se definišu preci i potomci. Deskriptivni slotovi sadrže deklarativne informacije ili podatke (statičko znanje). Proceduralni dodaci sadrže funkcije koje mogu da usmeravaju proces rezonovanja (dinamičko znanje). Pokazivači na povezane frejmove/skriptove mogu se koristiti za transfer upravljanja na pogodniji frejm ili pokazivač na drugi slot u frejmu.

Faceti u slotovima mogu da budu sledećeg oblika:

- Eksplicitno zadata vrednosti (values)
- Pretpostavljena vrednost (default) - koristi se ako nije zadata druga vrednost
- Opseg (range) - koja vrsta informacija može da se pojavi u slotu
- if-added - proceduralni dodatak koji specificira akciju koja će biti preduzeta ako se vrednost slota doda ili modifikuje (data-driven, event-driven, bottom-up rezonovanje)
- if-needed - proceduralni dodatak koji aktivira proceduru koja spolja pribavlja informacije koje slot nema (expectation-driven; top-down rezonovanje)
- if-removed - procedura koja će se izvršiti kada se vrednost ukloni iz slota
- Ostalo - može da sadrži frejmove, pravila, semantičke mreže ili druge tipove znanja.

Prednosti frejma:

- Prilično intuitivan za manipulaciju
  - Sličan organizaciji znanja kod ljudi
  - Podesan za kauzalna znanja
  - Lakši za razumevanje od logike i pravila
- Vrlo fleksibilan

Nedostaci frejma:

- Sklonost da se frejmovi koriste za definicije koncepata
  - Nije prikladno jer može da postoji validna instanca koncepta koja se ne uklapa u stereotip
- Nasleđivanje
  - Nije uvek potrebno sva svojstva klasnih stereotipova propagirati na podklase
  - Promena slotova može da proizvede nenamerne posledice na podklasama

Logika izražava znanje u određenoj matematičkoj notaciji. Ako su tačne pretpostavke ili činjenice, tada su tačne i izvedene pretpostavke ili činjenice koje se dobijaju na osnovu pravila.

Logički jezici:

- Propoziciona logika
- Logika prvog reda
- Temporalna logika
- Teorija verovatnoće
- Fazi logika

Propoziciona logika određuje tačnost ili netačnost izjava (propozicija) oslanjajući se na njihovu matematičku predstavu i kombinujući ih sa operatorima/veznicima. To je jednostavan jezik koristan za predstavu ključnih ideja i definicija. Propozicionom logikom se samo ustanovljava postojanje ili nepostojanje fakta iz realnog sveta. Ima jednostavnu sintaksu i jednostavnu semantiku. Osobine:

- Deklarativna, delovi sintakse korespondiraju određenim činjenicama
- Kompozitna, značenje celine se određuje na osnovu značenja njenih delova
- Kontekstno nezavisna

Elementi:

- Logičke konstante - true, false
- Propozicioni simboli - P, Q, S, ...
- Zagrade – ()

Rečenice/hipoteze/formule nastaju kombinacijom prethodnih sa veznicima:

$\wedge$ i
$\vee$ ili
$\rightarrow$ implikacija
$\leftrightarrow$ ekvivalencija
$\neg$ negacija

Propoziciona signatura je neprazan skup simbola koji se zovu atomi. U propozicionoj logici baza znanja predstavlja se skupom formula. Logičko rezonovanje (prirodna dedukcija) se koristi kako bi se kreirale nove rečenice/formule koje logički proizilaze iz datog skupa formula.

Pravila rezonovanja u SBZ - rezonovanje u SBZ je ispravno (sound) ako svaka nova formula koja je nastala na osnovu zaključivanja je i logička posledica baze znanja (nikada se ne dolazi do zaključaka koji ne slede iz baze znanja). Rezonovanje u SBZ je kompletno (complete) ako se iz SBZ mogu proizvesti sve formule koje se logički mogu rezonovati iz baze znanja (pronalaze se svi zaključci koji sledi iz baze znanja). Pravila se uglavnom mogu podeliti u dve grupe:

- pravila uvođenja (leva kolona) – definišu kako da se *izvede* formula zadate sintaktičke forme
- pravila eliminacije (desna kolona) – pokazuju kako se *koriste* formule određene sintaktičke forme

Mane propozicione logike:

- Teško je identifikovati individuale
- Još teže je definisati svojstva individuala ili relacije između individuala
- Generalizacija, definisanje paterna ne može lako da se predstavi
- Nepraktična ako je potrebno opisati složenije situacije iz realnog sveta

Logika prvog reda pretpostavlja da se svet sastoji od objekata, relacija i funkcija. Proširuje propozicionu logiku sa predikatima (je_ptica, ima_krila) i kvanifikatorima (za svako, postoji). Logika prvog reda dozvoljava definisanje kvantifikatora za individuale, ali ne i za predikate i funkcije. Signatura se sastoji od dva skupa simbola – funkcijskih (sqrt, length) i predikatskih. Svaki od ovih simbola ima dodeljenu arnost (arity) – relaciju između n elemenata. Predikatski simboli arnosti 0 su propozicione konstante. Funkcijski simboli arnosti 0 su objektne konstante. Objektne varijable su elementi koje čine nizovi objektnih simbola. Formule logike prvog reda formiraju se od atomičnih formula upotrebom propozicionih veznika i kvantifikatora ∀ i ∃. Kao i propoziciona logika i ovaj sistem je ispravan i kompletan. Dodatni koncepti u odnosu na propozicionu logiku:

- kompleksni objekti (termovi)
- relacije (predikatske i funkcijske)
- sintaksa
- semantika
- pravila rezonovanja
- korišćenje

Objekti mogu biti entiteti iz realnog sveta, često korišćeni koncepti (boja, težina) ili svojstva. Relacije ostvaruju konekciju između objekata. Njihova semantika zavisi od objekta nad kojima se ona primenjuje. Funkcije su specijalni tip relacija. Sintaksa se bazira na rečenicama/izjavama/funkcijama koje su dosta kompleksnije nego u propozicionoj logici. Sastoji se od konstantnih, predikatskih i funkcijskih simbola. Semantika pruža interpretaciju za bazične konstrukte. Termovi su logički izrazi koji definišu objekte. Konstante i objektne varijable su termovi. Kompleksni termovi se konstruišu od funkcijskih simbola i jednostavnih termova koji se uokviruju zagradama. Termovi signature σ su sačinjeni od objektnih varijabli, objektnih konstanti i funkcijskih simbola za σ. Termovi su jednaki za datu interpretaciju akko predstavljaju isti objekat.

Ograničenja logike prvog reda:

- Nije moguće predstaviti tranzitivno zatvaranje
- Ukoliko imamo funkcijske simbole veće arnosti od 0 ne možemo da damo opštu specifikaciju zatvaranja domena

Logika drugog reda obogaćuje sintaksu funkcijskim i predikatskim varijablama arnosti n>0. Funkcijske varijable mogu se koristiti za formiranje termova na isti način na koji se koriste i objektne varijable. Predikatske varijable se koriste za formiranje atomičnih formula na isti način na koji se koriste i predikatski simboli. U neatomičnim formulama, funkcijske i predikatske varijable vezuju se kvantifikatorima na isti način kao i objektne varijable.

Logički sistem je:

- **Ispravan (sound)** ako rezonovanjem nad netačnim iskazima nikada ne dolazimo tačnih zaključaka
- **Kompletan (complete)** ako se svi tačni iskazi mogu i dokazati
- **Odlučiv** (**decidable**) ukoliko postoji efikasna metoda koja (moguće je napisati program koji) određuje da li je formula teorema za zadatu bazu znanja.

Propoziciona logika (PL) je ispravna, kompletna i odlučiva, ali nije dovoljno ekspresivna za predstavljanje složenijeg znanja. Logika prvog reda (FOL) je ekspresivna, ispravna i kompletna, ali nije odlučiva. Deksriptivna logika (DL) je ekspresivnija od PL, ispravna, kompletna i odlučiva.

Deskriptivna logika je familija jezika za predstavljanje terminološkog znanja iz aplikacionog domena na strukturiran i formalno jasan/razuman način. Naziv deskriptivna (opisna) logika se koristi jer se domenski pojmovi predstavljaju pomoću opisa (deskripcija) koncepata. Deskripcije su izrazi sačinjeni od atomičnih koncepata (unarnih predikata) i atomičnih rola (binarnih predikata) korišćenjem konstrukta za povezivanje koncepata i rola. Konstrukti koji se koriste za pravljenje deskripcija zavise od konkretne logike. Razlikuje se od semantičkih mreža i frejmova po tome što ima striktno definisanu, formalnu, logičku semantiku. Opisi/deskripcije koncepta koriste za kreiranje iskaza koje čine bazu znanja u DL. Baza znanja najčešće ima dva dela: TBox – terminološki deo i ABox – činjenični deo.

TBox je terminološki deo baze znanja (šema). Opisuje relevantne pojmove iz aplikacionog domena tj. domena primene navođenjem svojstava koncepata i rola i veza među njima. Odgovara analogno šemi baze podataka. Najjednostavnija forma izraza u TBox bi bio iskaz kojim se uvodi naziv (skraćenica) koji se koristi za predstavu kompleksnih deskripcija koncepta. TBox imaju izražajnost/ekspesivnost. Odsustvo varijabli u opisima koncepata čini iskaze TBox-a čitljivijim od odgovarajućih iskaza logike prvog reda. TBox zovemo definicioni ako sadrži samo definicije koncepata. U definicionom TBox-u koncepti sa leve strane se zovu definisani koncepti, a koncepti sa desne strane primitivni koncepti.

Činjenice (asertacioni deo baze znanja) zovu se ABox (podaci). Opisuju konkretne situacije iz aplikacionog domena navođenjem vrednosti individuala i njihovih svojstava. ABox odgovara analogno podacima u bazi podataka.

Rezonovanje u DL sistemima - SBZ koji koriste DL omogućuju automatsku dedukciju implicitnog znanja iz eksplicitno predstavljenog znanja. Garantuju tačan odgovor u konačnom vremenu. Prilikom rezonovanja u obzir se uzimaju i terminološki iskazi (šema, TBox) i asertacioni iskazi (podaci, činjenice, ABox). Algoritmi rezonovanja:

- Algoritam obuhvatanja
- Algoritam instanciranja
- Algoritam konzistentnosti

Algoritam obuhvatanja (subsumption) određuje relacije podkoncept/nadkoncept. Ako koncept D (nadkoncept) obuhvata koncept C (podkoncept), onda su sve instance koncepta C ujedno i instance koncepta D. Deskripcija koncepta C je podskup deskripcije koncepta D.

Algoritam instanciranja određuje relacije instanciranja. Individual *i* je instanca deskripcije koncepta C ako je *i* uvek interpretiran kao element interpretacije koncepta C.

Algoritam konzistentnosti proverava da li je baza znanja (ABox i TBox) u konzistentnom stanju (nije kontradiktorna).

Zahtevi od DL sistema:

- Da bi se osiguralo razumno i prediktivno ponašanje DL sistema, svi problemi koji se rešavaju treba da budu **odlučivi (decidable)**
- Da bi DL sistemi bili upotrebljivi u realnim situacijama, problemi koji se rešavaju treba da budu **niske složenosti**
- DL sistemi moraju da budu **dovoljno ekspresivni** da bi mogli da se koriste za opis kompletnog domena primene
- Potrebno je napraviti dobar **kompromis između ekspresivnosti i složenosti problema**

ALC (***A***ttributive concept ***L***anguage with ***C***omplements) je primer bazičnog DL jezika. Deskriptivna logika koja podržava: konjunkciju, disjunkciju, negaciju, egzistencijalna ograničenja i ograničenja vrednosti.

Semantički web je formalno reprezentovanje znanja na webu i rezonovanje nad tim formalno reprezentovanim znanjem. To je web koji razumeju i računari, a ne samo ljudi. Namenjen je softverskim agentima. Definisan je strukturiranim dokumentima (npr. ontologijama) koji mogu da referenciraju jedan drugog i da formiraju na taj način ogromne mreže. Koristi se za simulaciju znanja u računarskim sistemima. Jezik za reprezentovanje znanja je OWL. Semantika OWL se definiše prevođenjima u ekspresivnu deskriptivnu logiku. Baza znanja OWL ontologija odgovara DL TBox-u sa hijerarhijom rola. Ontologija se sastoji od skupa asertovanih aksioma.

Osobine dobrih RZ jezika (jezika za reprezentaciju znanja):

- izražajni
- koncizni
- nedvosmisleni
- nezavisni od konteksta - što se iskaže danas moći će da se interpretira sutra
- efikasni - znanje se može predstaviti u mašinski čitljivom obliku, za odabrani format postoje praktične procedure zaključivanja
- efektivni - postoji procedura zaključivanja kojom se iz reprezentacije znanja mogu napraviti nove rečenice

Ontologija je formalno imenovanje i definisanje tipova, svojstava i veza entiteta koji stvarno ili fundamentalno postoje u određenom domenu diskursa. Ontologija je formalna specifikacija deljene konceptualizacije. Njena svrha je uspostavljanje zajedničkog rečnika za domen. Obezbeđuje semantiku za koncepte. Reči se koriste kao deskriptori koncepata. Više ontologije definišu osnovne, apstraktne koncepte. Niže ontologije definišu domenski specifične koncepte. Meta ontologije definišu druge ontologije.

Jena je Java API pomoću koga možemo da kreiramo i manipulišemo RDF grafovima. Ima klase i interfejse koje predstavljaju grafove, resurse, svojstva i literale. Vrednost svojstva može da bude i resurs osim literala. Svaki povezani par čvorova u RDF grafu je iskaz i predstavlja tvrđenje činjenice o resursu. Ima tri elementa: subjekat (resurs od koga polazi luk u grafu), predikat (labela luka u grafu) i objekat (literal ili objekat ka kome ide luk u grafu). S obzirom da ima tri elementa, naziva se još i triplet. RDF graf je skup iskaza. Klase, svojstva i individuali u RDF su predstavljeni kao URI. Jena omogućuje postavljanje upita nad modelima. Jena omogućuje standardne operacije nad RDF grafom: unija, presek i razlika. Pomoću ontologije možemo na formalan način da predstavimo značenje koncepata i relacija koji zajedno karakterišu neki domen. Tipična greška je da se ontologija koristi umesto baze podataka. Graf je interni Jena interfejs koji predstavlja skup iskaza. Rezoner koristi sadržaj osnovnog grafa i semantička pravila da kreira skup osnovnih i izvedenih iskaza. Ovaj skup iskaza je takođe graf. **OntModel** klasa koristi taj spoljašnji graf. Pored navođenja osobina koje individual treba da zadovolji da bi pripadao klasi (**intenzija**), OWL predviđa mogućnost definisanja klase nabrajanjem individuala (**ekstenzija**).

U ontologiji, svojstvo je naziv relacije između dva resursa ili između resursa i literala. Svojstva nisu definisana nad klasama već su first-class objekti sami. Aplikacije koje koriste ontologije mogu direktno da skladište, pronalaze i postavljaju iskaze o svojstvima. Jena Ontology API nudi set klasa i metoda namenjenih za rukovanje svojstvima u ontologiji. U Jena Ontology API postoje tri interfejsa koja nasleđuju OntProperty: **ObjectProperty**, **DatatypeProperty** i **AnnotationProperty**. Ovi interfejsi nemaju različita ponašanja. Object i Datatype svojstva mogu da budu funkcionalna - za jednu vrednost u domenu biće tačno jedna vrednost u opsegu. Podatak o tome da li je svojstvo funkcionalno predstavlja se kao facet resursa.

Pomoću restrikcije definišemo klasu tako što navodimo ograničenje koje individuali te klase imaju na nekom svojstvu. Postoji šest vrsta restrikcija u OWL jezicima.

Restriction	**Meaning**
value	The restricted property has exactly the given value.
all values from	All values of the restricted property, if it has any, are members of the given class.
some values from	The property has at least one value which is a member of the given class.
cardinality	The property has exactly *n* values, for some positive integer n.
min cardinality	The property has at least *n* values, for some positive integer n.
max cardinality	The property has at most *n* values, for some positive integer n.

Unija, presek i komplement su nam potrebni da bismo mogli da kreiramo restrikcije nad klasama.